YEM MART — Phase 1: Full Project Audit

Read-only audit of the uploaded ZIP (`yem-mart-_-your-everyday-store__5_.zip` , 47 files). Every finding below comes from files I opened in this session. No source file was modified. Secret values are redacted. I did not run `npm install` , `tsc` or `vite build` (no `node_modules` , network disabled), so build health is **unverified**.

Two notes up front

- The **12-phase roadmap was not supplied** in this conversation or inside the ZIP (no roadmap/phase text in any file). Task 4 (roadmap comparison) is therefore **not done**. Please send the roadmap and I will map it.
- This ZIP is byte-identical to the copy I examined earlier in this chat, so earlier line numbers were re-checked against it and still hold.

1. Project architecture summary

Area	Finding (source)
Framework	React 19 SPA, Vite, Tailwind 4 (`package.json` , `vite.config.ts`)
Backend	One Express 4 app in `server.ts` (1,070 lines), state in `server/store.ts` (1,065 lines)
Runtime	Node ES modules run with `tsx` (`start` / `dev` = `tsx server.ts`); `build` = `vite build` only
Database	Supabase Postgres via `server/supabase.ts` (service-role key, silent fallback to anon key, line 4); local JSON file `data/yem_database.json` as fallback/cache (`store.ts:28-29`)
Data model	In-memory object per server instance, re-hydrated from Supabase at most every 30 s (`server.ts:717-720`)

Area	Finding (source)
Deployment	Vercel: static Vite build + one serverless function `api/index.ts` that re-exports the Express app; rewrites in `vercel.json` send `/api/*`, `/product/*`, `/category/*`, `/sitemap.xml`, `/robots.txt` to it
AI	Gemini chatbot at `POST /api/chat` (`server.ts:913-988`)
Payments	Manual: website order ID → customer sends WhatsApp → pays by UPI → types UTR → admin verifies by hand. No gateway, no COD code
Package manager	`bun.lock` present but README says `npm`; no `engines` field

Where Supabase is defined: only `server/supabase.ts` (createClient at lines 16-22). `lib/supabase.ts` and `server/lib/supabase.ts` are one-line re-exports. The frontend never talks to Supabase directly. Tables touched by code: `store_settings`, `products`, `coupons`, `orders`, `stock_adjustment_logs`, `audit_logs`.

Environment variables used in code (names only): `ADMIN_PASSWORD`, `ADMIN_AUTH_SECRET`, `GEMINI_API_KEY`, `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `NODE_ENV`, `PORT`, `VERCEL`. `.env.example` lists `APP_URL` (not used anywhere in code) and omits `ADMIN_AUTH_SECRET`.

2. Page and endpoint map

Customer UI (single page, modals, no router): home with hero carousel, category grid and product grids; product detail modal (also reachable at `/product/:id`, `App.tsx:176`); cart drawer; wishlist drawer; checkout modal (details → WhatsApp → UPI/UTR); My Orders modal; Order Tracking modal; Gemini chatbot; footer.

Admin UI: `/admin` (path check `App.tsx:114-132`) → login modal → `AdminPanel.tsx` (1,901 lines) with tabs dashboard, orders, products, inventory, coupons, audit, cms.

API endpoints (`server.ts`)

Method + path	Line	Guard
GET `/sitemap.xml`, `/robots.txt`	53, 91 (duplicates at 677, 668)	public
GET `/product/:id`, `/category/:slug` (+ `/*`)	414-417	public (SEO HTML)
POST `/api/admin/login`	551	public
POST `/api/admin/logout`, GET `/api/admin/me`	640, 649	token
GET `/api/health`	663	public
GET `/api/products`, `/api/products/:id`	715, 731	public (drafts hidden unless admin)
POST `/api/products`, bulk-import	741, 757	any admin role
DELETE `/api/products/:id`	750	owner/manager
POST `/api/cart/validate`	768	public
GET `/api/orders`, `/api/orders/:id`	779, 788	any admin role
POST `/api/orders`	795	**public**
PUT `/api/orders/:id/status`, `/verify-payment`	808, 818	any admin role
POST `/api/orders/:id/submit-utr`	826	**public**
POST `/api/orders/track`	837	**public**
POST `/api/inventory/adjust`; PUT `/api/settings`; coupons POST/DELETE	850, 869, 890, 896	owner/manager
GET `/api/settings`	862	public
POST `/api/store/rehydrate`	876	**public**
GET `/api/coupons`, `/api/audit-logs`, `/api/stock-logs`	886, 904, 908	any admin role
POST `/api/chat`	913	**public**

Order workflow (as coded): `validateCart` recalculates prices, coupon and delivery fee on the server (`store.ts:624-728`) → `createOrder` deducts stock immediately (`store.ts:752-835`) → status `whatsapp_pending` → customer sends WhatsApp → UPI payment → `submitUtr` (`store.ts:908`) → admin `verifyPayment` (`store.ts:890`) → status updates; cancellation restocks once (`store.ts:838-870`).

3. Existing features inventory (must not regress)

- Product catalogue with categories, search, sort, filters, featured/deal/new flags, variants, draft vs published
- Server-side cart validation: prices, stock, coupons (min purchase, usage limit), flat delivery fee with free-delivery threshold
- Server-created orders with generated IDs, stock deduction, idempotent restock on cancel
- Kerala-only delivery check by pincode and district lookup
- WhatsApp order message and receipt text; copy-to-clipboard
- UPI step with UTR submission; admin-only payment verification
- PDF invoice generation (`src/utils/pdfInvoice.ts`)
- My Orders (local history + server lookup) and Order Tracking with masked customer data
- Wishlist, cart persistence (localStorage)
- Admin console: orders, products (incl. bulk import), inventory adjustments with logs, coupons, settings, audit log
- HMAC-signed admin tokens with 8-hour expiry (`server.ts:440-487, 589-593`)
- Dynamic SEO for `/product/:id` and `/category/:slug` (meta, Open Graph, JSON-LD, canonical injection `server.ts:169-174`)
- Gemini chatbot with store-specific system prompt
- Global Express error handler returning JSON (`server.ts:989`)
- Explicit `.js` import extensions (I found no relative import without one)
- Mobile layout: bottom nav with My Orders, responsive grids, image fallback to a placeholder icon

4. Findings

Status key: **Confirmed** = I read the code that proves it. **Unverified** = depends on something I cannot see (live database, Vercel runtime, dependency installs).

CRITICAL

C1 — Hardcoded admin password and token bypass (Confirmed)

- `server.ts:420` defines a hardcoded fallback password (value redacted); `:421` uses it when `ADMIN_PASSWORD` is unset.
- `server.ts:445` accepts the raw password as a bearer token and returns an **owner** session, skipping the HMAC system. It accepts the hardcoded fallback **even when `ADMIN_PASSWORD` is set** to something else.
- `server.ts:566-568` accepts the fallback at login. `.env.example:19` repeats the value as a sample and says it is the default.
- Impact: anyone who has seen the source can send `Authorization: Bearer <fallback>` to every admin endpoint (orders with customer PII, settings, coupons, inventory).
- Fix: delete the constant; refuse to start (or refuse login) when `ADMIN_PASSWORD` is missing; remove branch 1 of `verifyAdminToken`; rotate the real password because the old value is in every copy of this ZIP.

C2 — Token-signing secret falls back to a public string (Confirmed)

- `server.ts:422`: `ADMIN_AUTH_SECRET || ADMIN_PASSWORD || '<constant>'`. With defaults, the secret is the hardcoded fallback, so anyone can forge a signed token for any role (`server.ts:471-484`).
- Fix: require a long random `ADMIN_AUTH_SECRET`, independent of the password, and fail closed.

C3 — Orders and products may not be saved to Supabase (code/schema mismatch Confirmed; live state Unverified)

- `store.ts:333-357` upserts `orders` with `customer`, `items`, `discount_amount`, `grand_total`, `status`, `status_timeline`. `supabase/schema.sql:136-160` defines `customer_name`, `phone`, `address`, `pincode`, `total`, `order_status` (several `NOT NULL`) and has none of the written columns.

- `store.ts:262-290` writes `original_price`, `stock`, `variants`, `discount_percent`; `schema.sql:43-90` has `mrp`, `stock_count`, `discount_percentage` and no `variants` column.
- supabase-js returns `{error}` instead of throwing; the `catch` blocks (e.g. `store.ts:358, 372, 389`) never inspect it, so failures are silent.
- Writes are not awaited by the request: `store.ts:833, 886, 904, 924` use `.catch(() => {})`. On Vercel the function can freeze after the response and drop the write.
- `store.ts:221` replaces `this.data.orders` wholesale with whatever Supabase returns, and `:169` does the same for products. Any order that never reached Supabase can vanish at the next hydrate.
- Audit and stock logs are written (`store.ts:433, 455`) but never read back in `hydrateFromSupabase`, so they reset on restart.
- Unverified: the live database may have been altered to match the code. Check the real column list in the Supabase Table Editor and place a test order before trusting persistence. A "connectivity OK" check does not prove writes succeed.

HIGH

H1 — Login lockout does not stop brute force (Confirmed) The password is checked (`server.ts:566`) before the lock check (`:618`), so a correct guess succeeds during lockout. The counter is keyed on the client-supplied `X-Forwarded-For` (`:436-438`), held in memory per instance (`:426-427`). Fix: check lock first, key on the platform-trusted IP, store counters durably, add delay.

H2 — CORS reflects any origin with credentials (Confirmed) `server.ts:22-36` and again `:993` echo the request `Origin` and set `Allow-Credentials: true`. Fix: allow-list your own domains.

H3 — Admin token stored in three client places, no revocation (Confirmed) `api.ts:11-55` writes `yem_admin_token` to sessionStorage, localStorage **and** a JavaScript-readable cookie; the server cookie (`server.ts:605`) lacks `HttpOnly` / `Secure`; `?admin_token=` query tokens are accepted (`:509`). Logout (`:640-647`) only deletes the in-memory entry; a signed token stays valid until expiry. Fix: HttpOnly Secure cookie, no query token, server-side revocation list or short expiry.

H4 — Weak role model (Confirmed) One shared password; the caller picks the role in the request body (`server.ts:555, 576`). Product create/bulk-import, order status and payment verification need only *any* admin role (`:741, 757, 808, 818`), so the "support"

and "fulfillment" roles can edit the catalogue and verify payments. Fix: per-role credentials and per-route role checks.

H5 — Public endpoints with no rate limiting (Confirmed) `/api/orders` (795), `/submit-utr` (826), `/track` (837), `/api/chat` (913, paid Gemini calls), `/api/cart/validate` (768) and `/api/store/rehydrate` (876, forces a full Supabase read; `?fresh=true` on `/api/products` and `/api/settings` does the same, `:716-720, 863-866`). No rate-limit code exists in the project.

H6 — Order lookup is easy to guess (Confirmed) IDs are `YEM-<date>-<4 random digits>` from `Math.random` (`store.ts:796-797`, ~9,000 per day). Tracking needs the ID plus only the **last 4 digits** of the phone (`store.ts:928-939`, `endsWith`). Without rate limiting, one known ID needs at most ~10,000 tries. Fix: longer random IDs (crypto), full phone match, rate limit.

H7 — UTR and status transitions are unguarded (Confirmed) `submitUtr` (`store.ts:908-925`) needs no auth and no state check: it can overwrite the UTR and reset status to `payment_submitted` on a verified, shipped or cancelled order. `verifyPayment` (`:890`) works without a submitted UTR. `PUT /status` (`server.ts:808-815`) accepts any non-empty status string. Fix: allowed-transition table and enum validation.

H8 — Customer input is not validated on the server (Confirmed) `POST /api/orders` passes `customer` straight into `createOrder` (`server.ts:795-803`, `store.ts:752`); only the cart is validated. The browser validates (`CheckoutModal.tsx:112-115`) but the server does not. Impact on tracking (`trackOrder` calls string methods on stored fields) is Unverified.

H9 — Customer data shipped in the repo (Confirmed) `data/yem_database.json` and `data/yem_database.backup.json` each hold 3 orders with name, phone, address and UTR fields. Neither is in `.gitignore`. The store also writes to this file at runtime (`store.ts:106-109`); whether that works on Vercel's read-only filesystem is Unverified.

H10 — Checkout hides failures (Confirmed) If order creation fails, `CheckoutModal.tsx:164-205` builds a local order with a client-made ID and proceeds to WhatsApp; the server never sees it. If UTR submission fails, `:274` only logs a warning and shows success. Admin will not see either.

MEDIUM

- **M1 — Duplicate and shadowed routes (Confirmed):** `/robots.txt` and `/sitemap.xml` are defined twice (`server.ts:53/91` and `677/668` ; the second pair is dead code). Static `public/robots.txt` and `public/sitemap.xml` also exist, hardcoded to `yemmart.in` . Vercel normally serves static files before rewrites, so the dynamic versions may never run on the live site (Unverified; test the live URLs). `server.ts:1035-1036` also serves `dist` for non-Vercel hosts.
- **M2 — Fabricated review count (Confirmed):** `server.ts:326` `reviewCount: product.reviewCount || 5` sends invented reviews in JSON-LD, a structured-data policy risk. Product pages use `og:type website` (`:335, 383`).
- **M3 — Domain mismatch (Confirmed in code):** `index.html:16-17` hardcodes `yemmart.in` and an Unsplash image; the site you gave me is `yemmart.vercel.app` . The homepage has no static canonical (injected later by `seo.ts:111`); no `<noscript>` fallback (0 matches).
- **M4 — Two hardcoded UPI handles (Confirmed):** `formatters.ts:103, 207` default to one handle; `initialData.ts:8` has a different one marked "PENDING". WhatsApp text and the checkout screen can show different payment IDs depending on settings load.
- **M5 — Stock is held before payment (Confirmed):** `createOrder` deducts stock at creation (`store.ts:759+`); nothing expires unpaid orders.
- **M6 — Per-instance memory state (Confirmed architecture):** the "concurrency-safe" comment (`store.ts:759`) holds only within one process; several serverless instances can oversell. Impact Unverified.
- **M7 — Gemini model name `gemini-3.5-flash` (`server.ts:956`) is Unverified;** confirm it exists for your key. No input-size or rate limit on chat.
- **M8 — Supabase key fallback (Confirmed):** `server/supabase.ts:4` silently uses the anon key if the service key is missing; with RLS enabled and no write policies, every write would then fail silently.
- **M9 — Schema over-grants (Confirmed):** `schema.sql:249-255` grants ALL on all tables to `anon` and `authenticated` . RLS (`:260-268`) is the only barrier; policies exist only for public reads. `orders` has no policy, which is correct, but a future mistake would expose PII.

LOW

- Unused schema tables `product_variants`, `order_items`, `order_status_history` (never read or written by code)
- `README.md` is AI Studio boilerplate; `clean` script references `server.js`; `APP_URL` unused; `bun.lock` vs npm ambiguity; no `engines`
- Client also generates order IDs with `Math.random` (`formatters.ts:11-13`)
- My Orders keeps order history and phone in localStorage (`yem_mart_my_orders`, `yem_mart_customer_phone`) with no notice to the customer
- Product images load from Unsplash; only one `loading="lazy"` in the app (performance)

5. Sensitive-file register (values withheld)

File	Contains
`server.ts:420, 422`	hardcoded admin password, hardcoded signing-secret fallback
`.env.example:19`	the same password as a sample value
`data/yem_database.json`, `data/yem_database.backup.json`	3 orders each: customer names, phones, addresses, UTRs
`src/utils/formatters.ts:103, 207`; `src/data/initialData.ts:8`	hardcoded UPI handles
`src/utils/formatters.ts` WhatsApp number	store WhatsApp number (public by design)
`public/robots.txt`, `public/sitemap.xml`	no secrets

No Gemini or Supabase keys are present in the ZIP (`.env*` is ignored by `.gitignore`).

6. Tests

Existing: none. No `*.test.*` / `*.spec.*` files, no test runner config, no `test` script. The only check is `lint: tsc --noEmit`, and I did not run it.

Missing (priority order)

1. Auth tests: raw-password bearer must return 401; forged token rejected; lockout works; role matrix per route
2. Unit tests: `validateCart` (price, stock, coupon limits, delivery threshold), `createOrder` stock deduction, cancel restock idempotency, status transitions, `trackOrder` privacy masking
3. API integration tests (supertest) for every route above, including malformed bodies
4. Supabase contract test against a **staging** project: write then read back an order and a product, assert no `{error}`
5. End-to-end (Playwright, 360×740 viewport): browse → cart → checkout → UTR → track → admin verify
6. SEO checks: sitemap/robots content on the deployed URL, JSON-LD validity, no invented ratings
7. Rate-limit and abuse tests for `/track`, `/submit-utr`, `/chat`

7. File-by-file change plan (proposal only; nothing changed)

File	Planned change	Findings
`server.ts`	remove fallback constants and raw-token branch; fail-closed env check; lock-before-compare; trusted IP; origin allow-list; HttpOnly cookie; drop query token; per-route roles; rate limiting; delete duplicate sitemap/robots; status enum and transition checks; protect `/rehydrate`; remove `	
`server/store.ts`	check `{error}` on every Supabase call and surface it; await order writes; non-destructive hydrate (merge, never delete unsynced orders); crypto order IDs; full-phone tracking; submit-utr/verify state guards; customer input validation; hydrate logs	C3, H6-H8

File	Planned change	Findings
`server/supabase.ts`	require the service-role key for server writes; no silent anon fallback	M8
`supabase/schema.sql`	align columns with code (or code with schema), tighten grants, add policies	C3, M9
`src/utils/api.ts`	token in memory plus HttpOnly cookie only	H3
`src/components/CheckoutModal.tsx`	remove fake local-order fallback; show errors on UTR failure	H10
`src/utils/formatters.ts`, `src/data/initialData.ts`	single UPI source from settings	M4
`index.html`, `public/*`	correct domain, remove static robots/sitemap or make them match	M1, M3
`.env.example`, `.gitignore`, `README.md`	remove password, add `ADMIN_AUTH_SECRET`, ignore `data/*.json`	C1, H9, L
`data/*.json`	remove from repo; keep as a private backup only	H9
new `tests/`	per section 6	all
`vercel.json`	verify function bundling and static precedence	M1

8. Deployment risks

- Rotating the admin password and secret is mandatory the moment C1 is fixed; the old value is in every shared ZIP
- Vercel filesystem is read-only except `/tmp`: JSON fallback writes may fail (Unverified)
- Product SEO routes read `index.html` at runtime (`server.ts:131`); `vercel.json` has no `includeFiles`, so the file may be missing in the function bundle (Unverified; view-source a live `/product/ID` page)

- Serverless cold starts re-hydrate from Supabase and drop unsynced memory (see C3)
- Dependencies not installed or resolved here: confirm `npm ci` and `vite build` pass in a clean environment before any phase ships
- Changing the schema on the live database needs a backup first

9. Backup strategy and safe implementation order

Backups before Phase 2

1. Keep the original ZIP read-only and record its SHA-256
2. `git init`, commit the untouched ZIP as the baseline, tag `phase1-baseline`; one branch per phase
3. Export Supabase (Table Editor CSV or `pg_dump`) and store it **outside** the repo; do the same for `data/*.json`
4. Create a separate **staging** Supabase project and a Vercel preview deployment; never test schema changes on production
5. Note the current Vercel production deployment so you can roll back by promoting it

Proposed order (to be aligned with your 12-phase roadmap once supplied)

1. Security hardening: C1, C2, H1-H4, with token rotation
2. Data layer: C3 on staging first (schema match, error checking, non-destructive hydrate)
3. Order-flow integrity: H5-H8, H10
4. Repo hygiene: H9, `.env.example`, `.gitignore`, README
5. SEO and routing cleanup: M1-M4
6. Test suite built alongside steps 1-5, not after
7. Payments and conversion work (gateway, COD, policies, analytics) only after the above is stable

Stopping here. I have not started Phase 2. Reply with approval (and the 12-phase roadmap) and tell me which findings to implement first.